

ENGENHARIA DE SOFTWARE

Modelos de Ciclo de Vida de Software

Prof. Me. Valdir Prado · Disciplina: Engenharia de Software



Radiografia da Turma: 12 Alunos

77,8%

Experiência Acadêmica

Somente projetos teóricos — ninguém com vivência sênior no mercado.

55,6%

Domínio dos Ciclos

Classificaram o próprio conhecimento no nível 3 de uma escala de 5.

77,8%

Sem Métricas de Custo

Nunca estudaram ou atuaram com estimativas de custos ou métricas.

📌 **Principais desafios citados:** Falta de clareza nos objetivos de negócio, análise de requisitos e transformar o desenvolvimento em fluxo previsível sem sacrificar agilidade e qualidade.



O Que é um Modelo de Ciclo de Vida?

Um processo de software é um conjunto de atividades relacionadas que levam à produção de um produto. **Não é sobre sair digitando código aleatoriamente.**



Assim como na construção civil: você não levanta paredes sem sondagem do solo, projeto do arquiteto e cálculo estrutural.

Na Engenharia de Software, escolher o processo correto define se a empresa de TI terá lucro ou vai à falência.



PREMISSA FUNDAMENTAL

A Falsa Ilusão do Processo Universal

Se algum consultor disser que existe um processo universal de Engenharia de Software aplicável a tudo — **demita esse consultor.**

Sistema Crítico (ROM)

Controle embutido em motor de carro: alteração futura é quase impossível e caríssima. Exige ciclo de vida **rigoroso** com análise completa prévia.

Sistema Web (E-commerce)

Plataforma de vendas online: exige esteira **ágil e iterativa** para testar conversão de vendas rapidamente.

Qual modelo de ciclo de vida eles escolheriam para o software de uma UTI neonatal?



Justifiquem tecnicamente em uma frase

Modelo Cascata (Waterfall)



Dirigido a Planos

Todas as atividades devem ser planejadas e programadas antes de começarem. O resultado de cada estágio é a aprovação de documentos.

Onde é Usado?

Sistemas críticos de segurança: controle de voo de um Boeing, software de banco multinacional, freio ABS. **Não toleram falhas em produção.**

⚠ Analogia: descer uma corredeira de rio — a água que passou não volta mais.

Modelo Cascata (Waterfall)

Como vimos, o Cascata exige que o processo seja linear, onde o estágio seguinte não deve ser iniciado até que a fase anterior seja concluída e aprovada.

Vantagens	Desvantagens
A grande força do Cascata é que o processo se torna visível por meio da documentação produzida em cada fase. Isso permite que os gerentes possam monitorar o progresso de acordo com o plano de desenvolvimento inicial. Ele é excelente quando os requisitos são perfeitamente compreendidos desde o primeiro dia e têm pouca chance de mudar.	A sua divisão inflexível em estágios distintos é o seu maior problema. Nós somos forçados a assumir compromissos em um estágio muito inicial do processo, o que dificulta o atendimento às mudanças de requisitos dos clientes que surgem depois. O congelamento prematuro dos requisitos pode nos levar a entregar um sistema que o usuário não quer mais



ABORDAGEM MODERNA

Desenvolvimento Incremental

O **Desenvolvimento Incremental** trata da *adição de partes*. O sistema é desenvolvido como uma série de

versões (Incrementos), de maneira que cada versão

Desenvolve-se uma implementação inicial, expõe-se aos usuários e cria-se várias versões. Especificação, desenvolvimento e validação são **intercalados**, não separados.

Vantagem Comercial

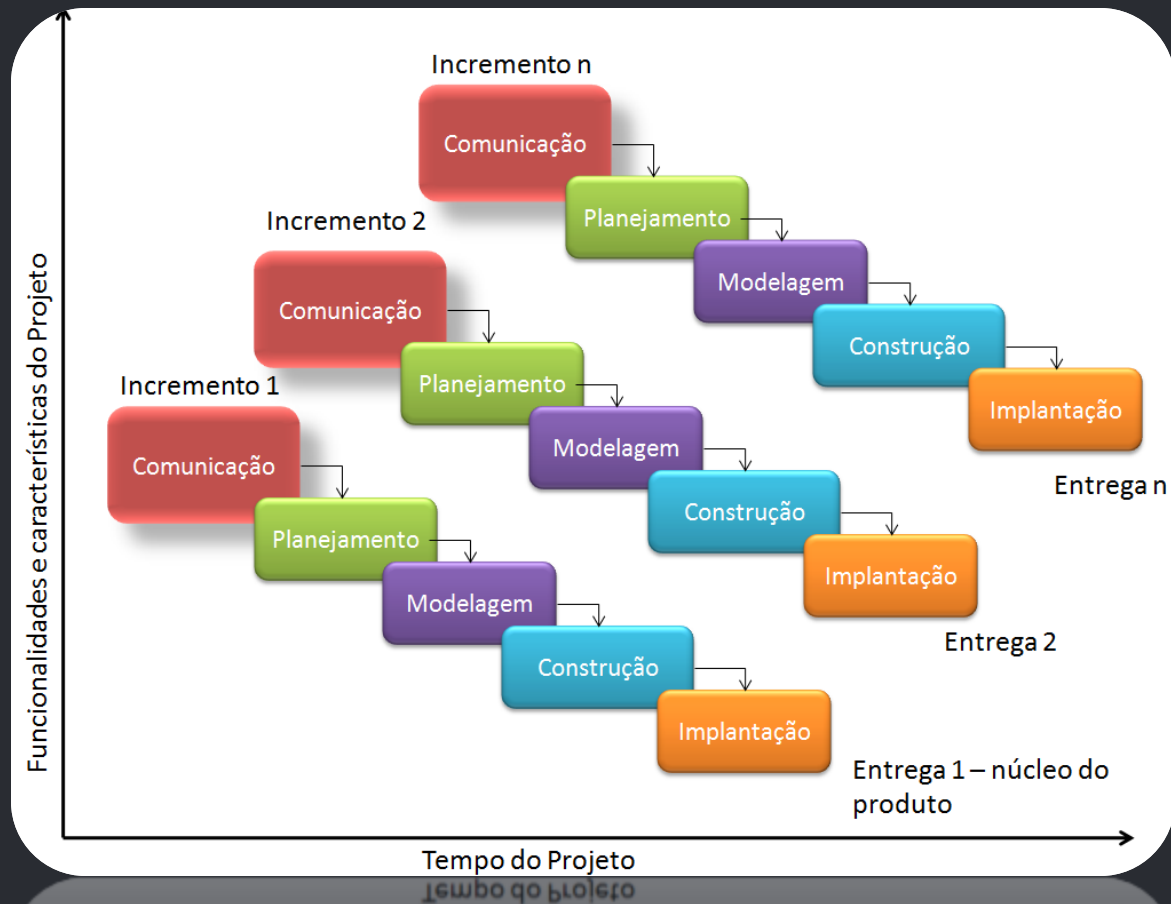
Entrega rápida de software útil. O cliente começa a ganhar dinheiro **meses antes** do projeto inteiro terminar.

Risco Gerencial

O processo é menos visível. Sem disciplina de refatoração, a estrutura do sistema **tende a se degradar**.

Desenvolvimento Incremental

O Desenvolvimento Incremental trata da adição de partes. O sistema é desenvolvido como uma série de versões (incrementos), de maneira que cada versão adiciona funcionalidade à anterior.



A Analogia: Imagine pintar um quadro por incrementos. Primeiro, eu pinto apenas o céu e entrego a tela para você. Na semana seguinte, eu pinto as montanhas. Na outra semana, a casa. Eu estou construindo a solução final pedaço por pedaço. Em software, seria eu entregar a tela de login na semana um, o carrinho de compras na semana dois e o gateway de pagamento na semana três.

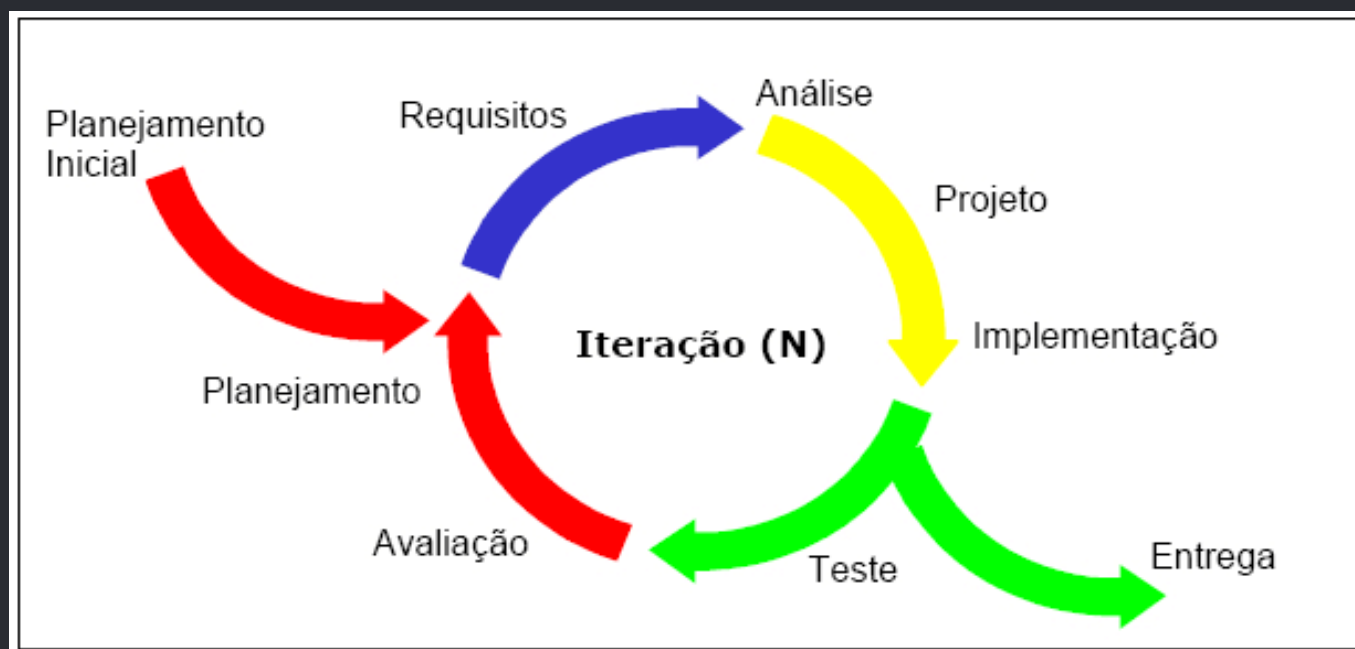
Desenvolvimento Incremental

Como vimos, o Cascata exige que o processo seja linear, onde o estágio seguinte não deve ser iniciado até que a fase anterior seja concluída e aprovada.

Vantagens	Desvantagens
O custo financeiro de acomodar as mudanças nos requisitos do cliente é drasticamente reduzido, pois a quantidade de análise e documentação a ser refeita é muito menor. Além disso, é mais fácil obter feedback contínuo, e nós conseguimos uma entrega e implementação rápida de um software útil ao cliente antes do fim do projeto.	Do ponto de vista gerencial, o processo não é tão visível, pois não é economicamente viável produzir documentos extensos para cada pequena versão que lançamos. O maior perigo técnico é que a estrutura do sistema tende a se degradar com a adição contínua de novos incrementos. Se a nossa equipe não gastar tempo com a refatoração do código, as mudanças futuras se tornarão caras e complexas.

Modelo Iterativo

O **Desenvolvimento Iterativo** trata do *refinamento e retrabalho*. O processo volta às mesmas partes do sistema para melhorá-las repetidas vezes (iterações).



A Analogia: Agora, imagine pintar o mesmo quadro de forma iterativa. Na primeira iteração, eu faço um esboço inteiro a lápis de tudo (céu, montanhas e casa). Você olha e aprova a disposição. Na segunda iteração, eu pinto tudo com cores básicas, em aquarela. Na terceira, eu coloco as sombras e os detalhes finos com tinta a óleo. Eu não entreguei pedaços isolados; eu entreguei o todo em versões cada vez mais refinadas e detalhadas.

Desenvolvimento Iterativo

Adotar o Modelo Iterativo puro ou combinado significa aceitar que não vamos acertar tudo de primeira. Nós vamos construir a solução completa de forma superficial, avaliar e refinar repetidas vezes. Isso traz consequências drásticas, tanto positivas quanto negativas, para o nosso fluxo de trabalho.

Vantagens

Mitigação Precoce de Riscos: Como passamos por todo o ciclo do sistema em cada iteração (do esboço aos detalhes finais), nós descobrimos problemas arquiteturais e de negócio graves logo no início, antes de comprometermos todo o orçamento.

Feedback Constante e Alinhamento: O cliente não precisa tentar imaginar o software lendo um documento de 500 páginas. Ele vê a evolução do produto como um todo a cada ciclo e pode ajustar a nossa rota de desenvolvimento.

Qualidade e Refinamento: A cada passagem pelo código, a nossa equipe tem a oportunidade técnica de otimizar o desempenho e melhorar a usabilidade da interface, entregando um produto final muito mais maduro.

Desenvolvimento Iterativo

Adotar o Modelo Iterativo puro ou combinado significa aceitar que não vamos acertar tudo de primeira. Nós vamos construir a solução completa de forma superficial, avaliar e refinar repetidas vezes. Isso traz consequências drásticas, tanto positivas quanto negativas, para o nosso fluxo de trabalho.

Desvantagens

Complexidade de Gestão: É extremamente difícil para um gerente de projetos aprovar um cronograma e um orçamento fixos quando a própria natureza do modelo exige que voltemos e refaçamos partes do sistema várias vezes.

Síndrome da Obra Inacabada (*Scope Creep*): Sem uma liderança técnica forte impondo limites, o cliente sempre pedirá 'só mais um pequeno ajuste'. O projeto pode entrar em um ciclo infinito de polimento, estourando todos os prazos.

Desgaste do Cliente: Exigir que a diretoria ou o cliente avalie e valide o sistema inteiro a cada nova iteração consome muito tempo e energia do lado do negócio. É comum o cliente perder o engajamento e parar de responder ao longo dos meses.

Desenvolvimento Iterativo

No mercado de TI, nós não deixamos a iteração rodar solta. Nós usamos métricas rígidas de aceitação para dizer: 'Este refinamento atende à necessidade atual. Não está perfeito, mas está pronto para ir para produção'. A perfeição excessiva é a maior inimiga da inovação.

O Custo do Retrabalho: Prevenção vs. Tolerância

Lei imutável de TI: os requisitos sempre vão mudar. A mudança aumenta exponencialmente os custos porque gera retrabalho.

Prevenção de Mudanças

O processo antecipa mudanças antes do retrabalho. Exemplo: protótipo de interface para validação do cliente.

Analogia: maquete 3D hiper-realista antes do primeiro prego.

Tolerância a Mudanças

O processo é projetado para que alterações tenham custo baixo. Exemplo: entrega incremental.

Analogia: divisórias de drywall — mudar salas custa mínimo esforço.

Modelos Evolutivos (A Força da Prototipação)

O PERIGO DO PROTÓTIPO DESCARTÁVEL: A interface funciona, mas a fundação de software não existe!



Os modelos evolutivos atacam frontalmente a incerteza. Um protótipo é uma versão inicial de um sistema, usado para demonstrar conceitos e experimentar opções de projeto, não sendo o produto final.

Modelos Evolutivos (A Força da Prototipação)



A Armadilha do Mercado: O perigo real que vocês enfrentarão nas empresas é a pressão gerencial para entregar o protótipo como produto final, o famoso protótipo descartável. O diretor vê a tela funcionando e diz: 'Já está pronto, lance para os clientes!'. Qual é o desastre aqui? É impossível ajustar esse protótipo para atender aos requisitos não funcionais, como desempenho, proteção, robustez e confiabilidade, que foram ignorados durante o seu desenvolvimento rápido. A estrutura do sistema já nasce degradada, sem documentação, e a manutenção futura será o caos.

Modelos Evolutivos (A Força da Prototipação)

O foco da prototipação é desenvolver rapidamente uma versão inicial para demonstrar conceitos e descobrir mais sobre o problema antes de construir o software real.

Vantagens	Desvantagens
Ela é a melhor ferramenta para revelar erros e omissões nos requisitos. Os usuários podem experimentar a interface, descobrir pontos fortes e fracos e validar se o sistema realmente apoiará o trabalho deles.	O grande risco de mercado é a pressão gerencial para transformar um 'protótipo descartável' em um sistema de produção. Como o foco foi a rapidez, esse código não atende aos requisitos não funcionais, como desempenho, proteção e confiabilidade. Além disso, a documentação é inexistente e os padrões de qualidade foram relaxados.

Modelos Modelo Espiral de Barry Boehm

O processo de software é representado como uma espiral, onde cada volta é uma fase iterativa. A principal característica é o reconhecimento explícito do risco.



Analogia: **Exploração de Marte** pela NASA. Cada volta mapeia ameaças: "E se o hardware falhar? E se perdermos o sinal?".



Usado em projetos de altíssimo risco tecnológico, como P&D de novos hardwares acoplados a softwares.



Engenharia de Software Orientada a Reúso

A realidade do mercado hoje é que não escrevemos todo o código do zero. A Engenharia de Software Orientada a Reúso depende de uma ampla base de componentes reusáveis e foca na integração desses componentes em um sistema já existente.



Análise de Componentes

Identificar e avaliar o que já existe.



Modificação de Requisitos

Adaptar às funcionalidades disponíveis.



Projeto com Reúso

Arquitetar a solução com peças existentes.



Desenvolvimento e Integração

Montar e conectar os componentes.

Imagine montar um computador de alto desempenho. Você compra o processador, a placa de vídeo e a memória, focando na montagem. Da mesma forma, hoje, um sistema de delivery usará um componente de mercado para o mapa e uma API financeira existente.



Este processo reduz custos, riscos e acelera o tempo de entrega, mas perdemos o controle sobre a evolução das versões desses componentes que não nos pertencem.

A Engenharia em "Brownfield" e Sistemas Legados

A academia muitas vezes ensina a criar sistemas no vácuo, o chamado projeto **Greenfield**, onde o terreno está limpo e pronto para construirmos do zero.

Mas a realidade corporativa é **Brownfield**. Sistemas de grande porte incluem e interagem com inúmeros bancos de dados e sistemas já existentes há décadas.

Em multinacionais, operadoras de telefonia ou grandes bancos, vocês raramente vão criar um sistema do zero. A maior parte dos requisitos estará preocupada com a interação e não se prestará facilmente à flexibilidade e ao desenvolvimento incremental.

- 📌 Analogia: Atuar em Brownfield é como tentar reformar e modernizar o motor de um avião comercial **enquanto ele está em pleno voo**. O novo componente não pode derrubar a operação principal.



Modelos Evolutivos, Reúso e Brownfield



Modelo Espiral (Boehm)

Cada volta da espiral: definição de objetivos, avaliação de riscos, desenvolvimento e planejamento. Usado em P&D de altíssimo risco tecnológico.



Engenharia Orientada a Reúso

Integra componentes de mercado (APIs, bibliotecas) em vez de criar do zero. Reduz custos e acelera entrega, mas perde controle sobre versões externas.



Brownfield Systems

Realidade corporativa: sistemas interagem com bancos e plataformas existentes há décadas. Reformar o motor de um avião **em pleno voo**.

RUP - Rational Unified Process

O RUP é um processo híbrido, projetado em conjunto com a UML, que reúne elementos de diversos modelos genéricos. Ele se destaca por não ter apenas uma visão temporal, mas por definir **fases de negócio** distintas que guiam o projeto.

1

Concepção

Estabelecer o caso de negócio e a visão do projeto.

2

Elaboração

Entender o problema, definir a arquitetura e planejar.

3

Construção

Desenvolvimento, programação e testes iterativos do sistema.

4

Transição

Entrega, implantação e transferência do sistema para usuários.

As atividades técnicas, como modelagem de requisitos, análise, projeto e testes, ocorrem em todas as fases de negócio, porém com intensidades diferentes. Empresas em setores governamentais e financeiros utilizam o RUP por seu equilíbrio entre **controle robusto e desenvolvimento iterativo**.

Inovações Contemporâneas no Ciclo de Vida

O cenário de desenvolvimento de software está em constante evolução, integrando novas tecnologias e abordagens que redefinem o ciclo de vida tradicional.



Desenvolvimento Assistido por IA

A IA acelera a codificação, mudando o gargalo para requisitos, arquitetura e segurança. Especificações imprecisas resultam em código incorreto de forma veloz.



Esteiras de Entrega Contínua

A cultura DevOps e a automação transformaram o ciclo em um fluxo contínuo. Pequenas atualizações diárias são validadas automaticamente antes da implantação.



Ciclos Híbridos (Microserviços)

Sistemas modernos raramente usam um único ciclo. Módulos críticos podem ter alta rigidez documental, enquanto interfaces de usuário evoluem em iterações rápidas semanais.



Engenharia de Plataforma

Criação de plataformas internas padroniza fluxos de trabalho e arquiteturas. Isso acelera o desenvolvimento, reduz a fricção e aumenta a confiabilidade dos sistemas.

RUP e Inovações Contemporâneas

RUP — Rational Unified Process

Processo híbrido com fases de negócio: **Concepção, Elaboração, Construção e Transição**. Atividades técnicas ocorrem em todas as fases com intensidades diferentes. Usado em setores governamentais e financeiros.

IA no Ciclo de Vida

Com IA gerando código em segundos, o gargalo migrou para **engenharia de requisitos e validação de segurança**. Especificação vaga = IA gera defeitos rapidamente.

Ciclos Híbridos e DevOps

Núcleo bancário segue modelo rigoroso; interface do usuário evolui em iterações semanais. Entrega contínua elimina a fronteira entre desenvolvimento e operação.



A Anatomia da Validação e os Estágios de Teste

Falhas de software podem quebrar empresas. Por isso, a **Validação** (ou Verificação e Validação - V&V) não é um detalhe, mas uma fase crítica e fundamental em qualquer ciclo de vida. Sistemas maduros não são testados como uma unidade monolítica apenas no final; o mercado exige três estágios de testes distintos.

1

Testes de Desenvolvimento (ou de Componente):

Cada componente é testado de forma independente para garantir que funciona conforme o esperado. É como testar cada parafuso de uma turbina.

2

Testes de Sistema

Os componentes são integrados e o sistema é testado como um todo para encontrar erros de interação e garantir a coesão. Seria como testar a asa acoplada à fuselagem.

3

Testes de Aceitação (Teste Alfa)

O sistema é testado com dados reais e cenários de uso fornecidos pelo cliente. Inclui os Testes Alpha (internos com o cliente) e Testes Beta (com potenciais usuários reais).

O custo de encontrar um bug na fase de desenvolvimento é insignificante; encontrá-lo na fase Beta, no entanto, pode comprometer a reputação da sua empresa e gerar perdas catastróficas.

A Anatomia da Validação e Estudo de Caso

4. Teste de Beta

Quando o software será comercializado como um produto para o mercado em geral, nós entregamos o sistema a um número de clientes potenciais que concordaram em testá-lo em seu ambiente real. Eles relatam os problemas encontrados antes de fazermos o lançamento oficial de vendas.



Estudo de Caso Guiado

Estudo de Caso — 4 grupos de 3 pessoas: Cada grupo recebe um cenário corporativo complexo. Missão: definir qual modelo de ciclo de vida salva o projeto e justificar tecnicamente. Tempo: 45 minutos de debate e formulação da estratégia.



Referências Bibliográficas

Bibliografia Básica:

- SOMMERVILLE, Ian. Engenharia de software. 9. ed. Pearson Addison Wesley, 2011.
- PRESSMAN, Roger S. Engenharia de software: uma abordagem profissional. 7ª ed. McGraw-Hill, 2011.
- ROBBINS, Stephen P. Comportamento organizacional. 11ª ed. Person Prentice Hall, 2005.

Bibliografia Complementar:

- SHORE, J.; WARDEN, S. A Arte do desenvolvimento ágil. Alta Books, 2008.
- REZENDE, D. A. Engenharia de software e sistemas de informação. Brasport 1999.